

16-Class Scene Recognition from 2,400 Images

Smriti Bhemireddy | ITCS 6169/8169 Computer Vision, Assignment 1 **GitHub:**

<https://github.com/smritib4/cnn-challenge-scene-classification>

1. Final Result

Test accuracy	96.75% (387 of 400), top-5 100%
Validation accuracy of selected model	95.63%
Model	ConvNeXt-Tiny, ImageNet-1k pretrained, all 27.8M parameters fine-tuned
Input	224 x 224 RGB, ~14 min on one Tesla T4
Reproduce	<code>python train.py --config configs/best.yaml</code>

The starter network gets 49.58% on the same split, so this is +47 points.

How I validated. I split the 2,400 images 80/20 with a stratified split at a fixed seed, so each class contributes exactly 30 of the 480 validation images, and I chose the model on validation accuracy alone under a rule set in advance: best validation accuracy among runs that finished. Stratification mattered because the starter's `random_split` gave per-class counts from 115 to 124, and on 480 images 1% is five images — as large as the differences I was trying to measure. I scored the test set on two selected checkpoints and report both: ResNet-50 got 94.25%, ConvNeXt 96.75%.

2. Secret Recipe

ConvNeXt-Tiny with ImageNet weights, fully fine-tuned, 224px colour, AdamW with the backbone at 1e-4 and the head at 10x, weight decay 0.05, cosine schedule with 3 warmup epochs, label smoothing 0.1, 40 epochs.

Almost all of the gain comes from two decisions, and neither is about architecture:

- **Pretrained features are worth ~32 points.** Freezing all of ResNet-18 and training

only the 16-way head — **8,208 trainable parameters** — already reaches 91.04%. My own CNN trained from scratch reaches 89.38% with 572x more parameters. At 1,920 images, where the features come from matters far more than what sits on top.

- **Resolution and colour are worth 8.8 points, with no change of model.** Giving the

starter's own network colour at 128px instead of grayscale at 64px takes it from 49.58% to 58.75%. Its preprocessing limited it before its architecture did.

Everything after that is small. Fine-tuning the backbone adds 3.4 points, and the whole range from ResNet-18 to ConvNeXt-Tiny (94.58% to 95.63%) is smaller than my noise floor, so I cannot claim ConvNeXt is the better architecture, only that it won the selection rule. Strong augmentation and weight EMA are in the final config because that is the run that was selected, not because I have evidence they helped.

3. Experimental Journey

Two numbers means I ran the same config twice at the same seed.

#	Experiment	Change	Params	Val acc (%)
00	Starter TNet, gray 64px	baseline	57.8K	49.58 / 49.58
01	+ colour, 128px	preprocessing only	246.5K	58.75 / 58.13
02	My own CNN, from scratch	no pretrained weights	4.7M	89.38
03	ResNet-18 linear probe	backbone frozen	8.2K	91.04 / 90.83
04	ResNet-18 fine-tuned	unfreeze	11.2M	94.58 / 93.96
05	ResNet-50 fine-tuned	model size	23.5M	95.21
06	+ strong augmentation	augmentation only	23.5M	94.38
—	Final: ConvNeXt-Tiny	family + EMA + flip TTA	27.8M	95.63

Every number in that column is **validation** accuracy, since that is what selection used. Only the two selected checkpoints were scored on the test set: experiment 5 got 94.25%, and the final model got **96.75%**.

The most useful thing I measured was my own noise floor. Running one config twice at the *same* seed gave 94.58% and 93.96% — 0.62 points apart, or 3 of 480 images. Same-seed GPU runs are not identical because cuDNN picks convolution algorithms by timing. That means any single-run difference under about 1 point here is not a result, which rules out most of my later comparisons and is why the table reports repeats instead of my best run.

4. Failure Analysis: strong augmentation

What I tried. Experiment 6 added RandAugment, colour jitter and RandomErasing to the ResNet-50 recipe, with 40 epochs instead of 30.

Why I expected it to work. 1,920 images against 23.5M parameters looked like textbook overfitting, and experiment 5 seemed to confirm it: validation accuracy peaked in the first couple of epochs, then drifted down while training loss kept falling.

What happened. 94.38%, against 95.21% for the same network with basic augmentation and ten fewer epochs — 0.83 points worse for a third more GPU time.

What I learned. An early peak followed by decline is not enough to diagnose the kind of overfitting augmentation fixes. The pretrained features were already close to right, so the useful part of training finished in two epochs and the rest was the head fitting noise. Augmenting pixels cannot help when the problem was never a lack of image variety. The loss curves would have told me first: training loss had already flattened at 0.568, essentially the floor label smoothing imposes, so the model was not struggling with the objective at all. My real mistake was ordering — I ran the comparison before measuring the noise floor, so a 0.62-point wobble looked like a trend.

5. AI + Human

Where it helped. I used Cursor. It wrote the config system, the code that swaps the classifier head across torchvision model families, and the plotting scripts. It also wrote a throwaway script that decompressed the assignment PDF's internal streams to recover the dataset link, which normal text extraction had missed.

Where I overruled it. Asked to download the dataset, it started a `gdown --folder` crawl fetching images one request at a time. The code ran fine, which is the problem: it was on track to take two hours. I stopped it, downloaded the folder as one archive, and put the reproducibility into `scripts/prepare_data.py`, which unpacks the archive and then *checks* it: per-class counts, class names matching across splits, and a warning if

the training total is not 2,400. I also refused the `random_split` the starter uses and the AI reproduced by default, for the reasons in section 1. Further cases are in `AI_USAGE.md`.